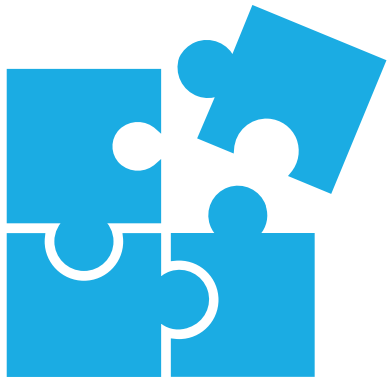




Introducción a React

SISTEMAS DISTRIBUIDOS E INTERNET

Vicente García Díaz
garciavicente@uniovi.es

Contenidos

Introducción

Prerrequisitos de trabajo

Hola Mundo

Conceptos JavaScript que usaremos

Componentes React que usaremos

Otras herramientas que usaremos

REACT

Introducción

¿Qué es React?

- React es una biblioteca JavaScript para crear interfaces de usuario
 - Es de código abierto
 - Licencia MIT
- Sitios oficiales
 - <https://react.dev/>
 - <https://github.com/facebook/react>
- No es un framework, es una biblioteca
- Ha sido creada por equipos de Facebook y de Instagram en 2013
- Algunos lo ven como la V de MVC



 **PayPal**

 **slack**



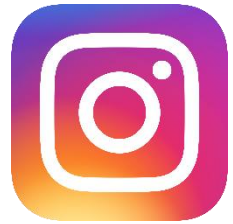
BBC

CNN

TESLA

vimeo

Uber **Eats**



The New York Times

 **shopify**



NETFLIX



 **BlaBlaCar**

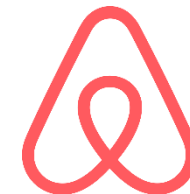
 **shopify**

Dropbox

codecademy

 **reddit**

 **BlaBlaCar**

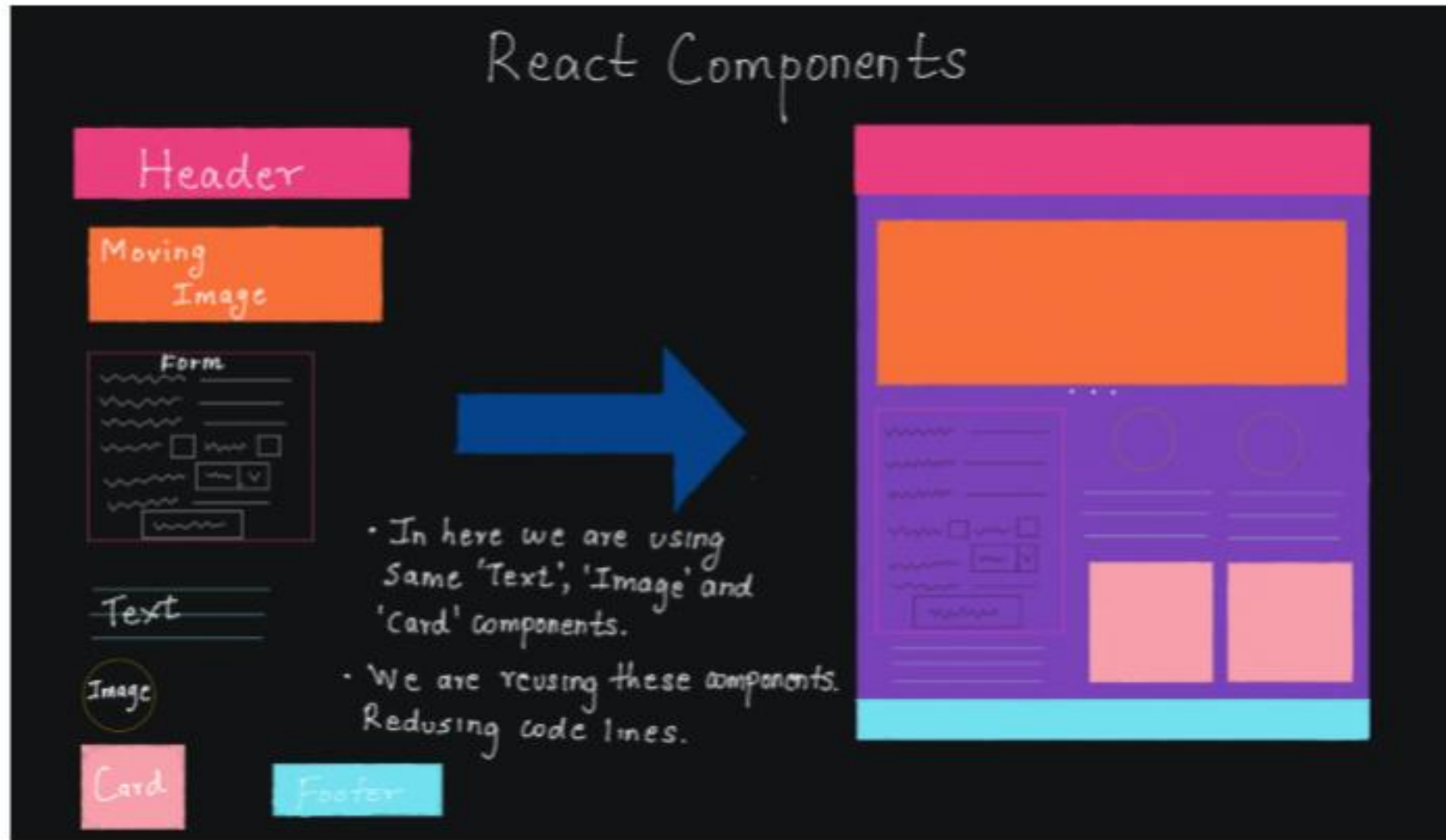
 **airbnb**

 **ATLASSIAN**

Algunas estadísticas

Angular	React	Vue
79.2K GitHub stars	181K GitHub stars	193K GitHub stars
20.8K Fork	36.9K Fork	31.3K Fork
1528 Contributors	1538 Contributors	404 Contributors
Usage - 2.1 million users	Usage - 8.8 million users	Usage - 3 million users

La clave son los componentes



<https://medium.com/nerd-for-tech/react-js-class-components-and-functional-components-f346cb92a02c>

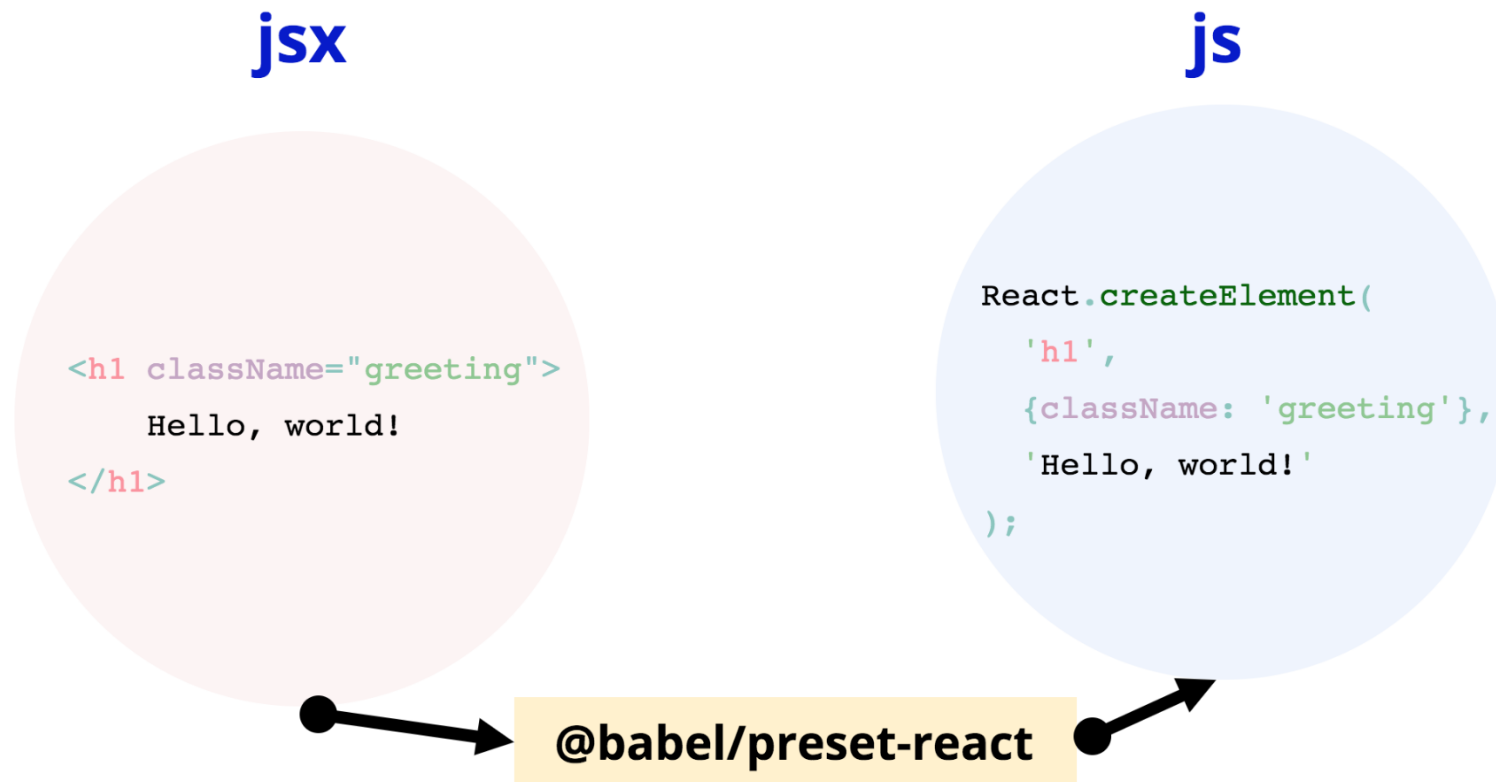
Componentes

- Los **componentes** son la parte más importante de una aplicación React
- Definen como se crean los elementos DOM y como los usuarios pueden interactuar con ellos
- Los componentes **son** archivos **JavaScript** que contienen código **HTML**
 - Sirven de plantillas
 - Sirven para incluir lógica
- React **utiliza** la **sintaxis JavaScript Extension Syntax (JSX)** para mezclar **JavaScript** y **HTML**

```
return (  
  <div>Hello World from React</div>  
);
```


Componentes (II)

- Internamente se usa **Babel** para convertir JSX a JavaScript



<https://jorgechavez.dev/2020/05/02/how-to-configure-react-app-with-webpack-and-babel/>

Componentes (III)

➤ Dos formas de escribir componentes en React

➤ Componentes funcionales

- Requieren menos código
- No tienen su propio estado

```
function App() {  
  return (  
    <div>hola</div>  
  );  
}
```

➤ Componentes de clase

- Requieren más código
- Tienen su propio estado

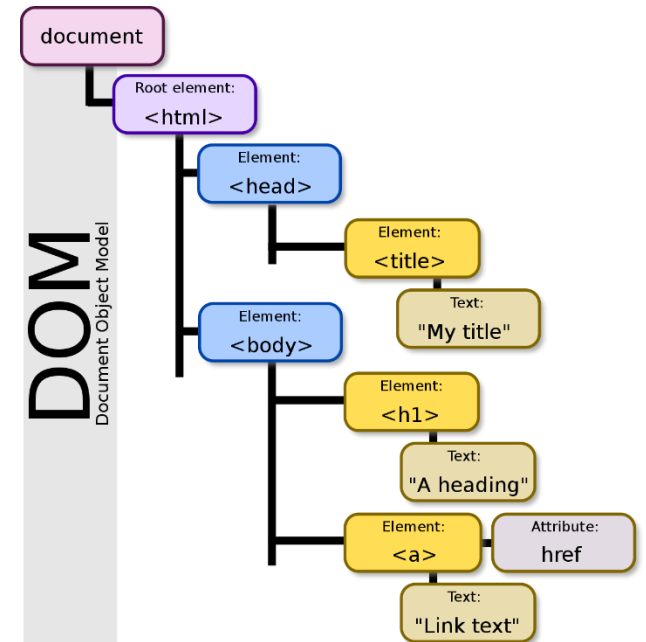
```
class App extends Component {  
  render() {  
    return (  
      <div>hola</div>  
    );  
  }  
}
```

Virtual DOM

- React mantiene una representación del DOM a la que llama **Virtual DOM (VDOM)**
- Gracias a esa representación las operaciones son muy rápidas
 - Realmente tiene 2 copias del VDOM
 - React calcula las diferencias entre las 2 copias del VDOM
 - Esas diferencias dan lugar a un conjunto de operaciones
 - Las operaciones se aplican al DOM
 - No es necesario cambiar el DOM completo, solo las operaciones indicadas por el VDOM

```
<ul>
  <li key="1">Skills</li>
  <li key="2">Projects</li>
</ul>

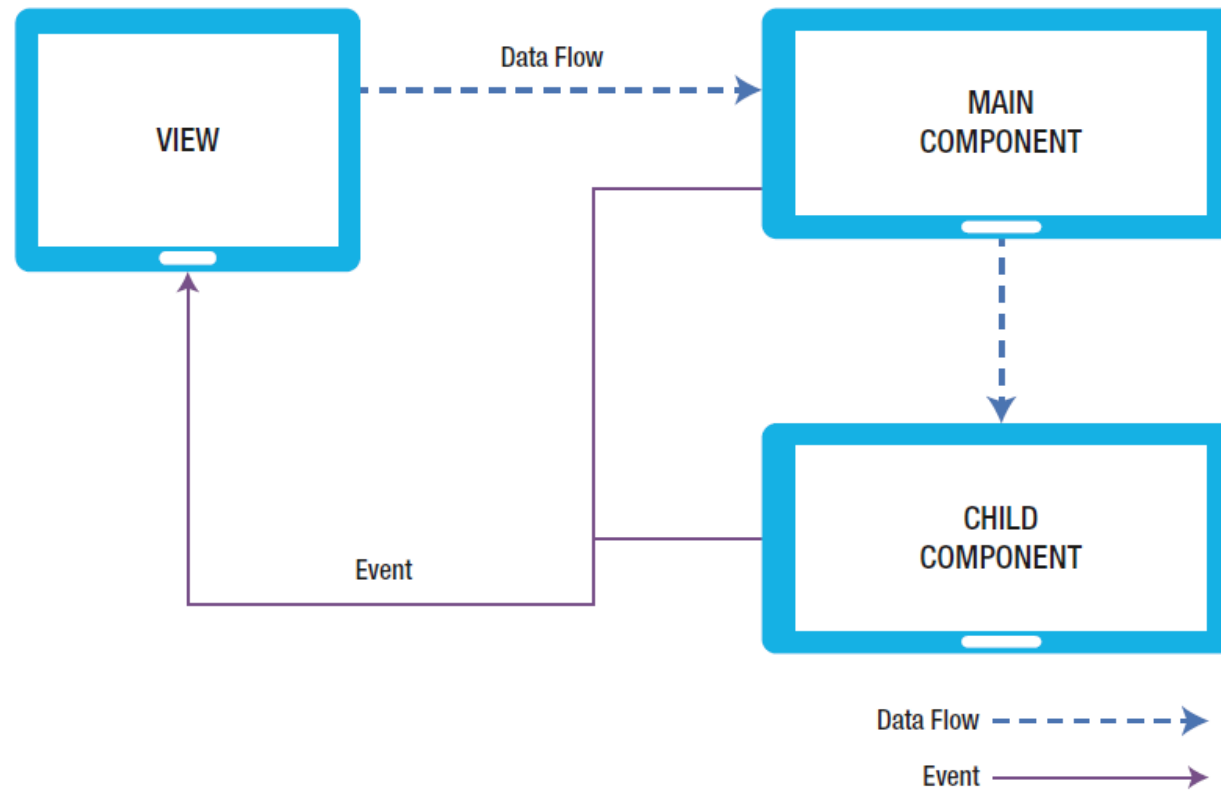
<ul>
  <li key="0">Flow</li>
  <li key="1">Skills</li>
  <li key="2">Projects</li>
</ul>
```



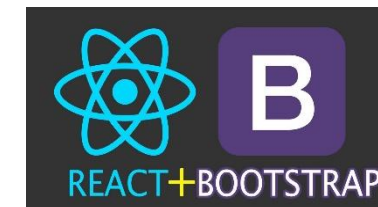
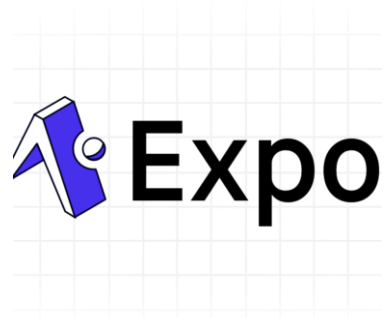
<https://commons.wikimedia.org/wiki/File:DOM-model.svg>

Flujo de datos unidireccional

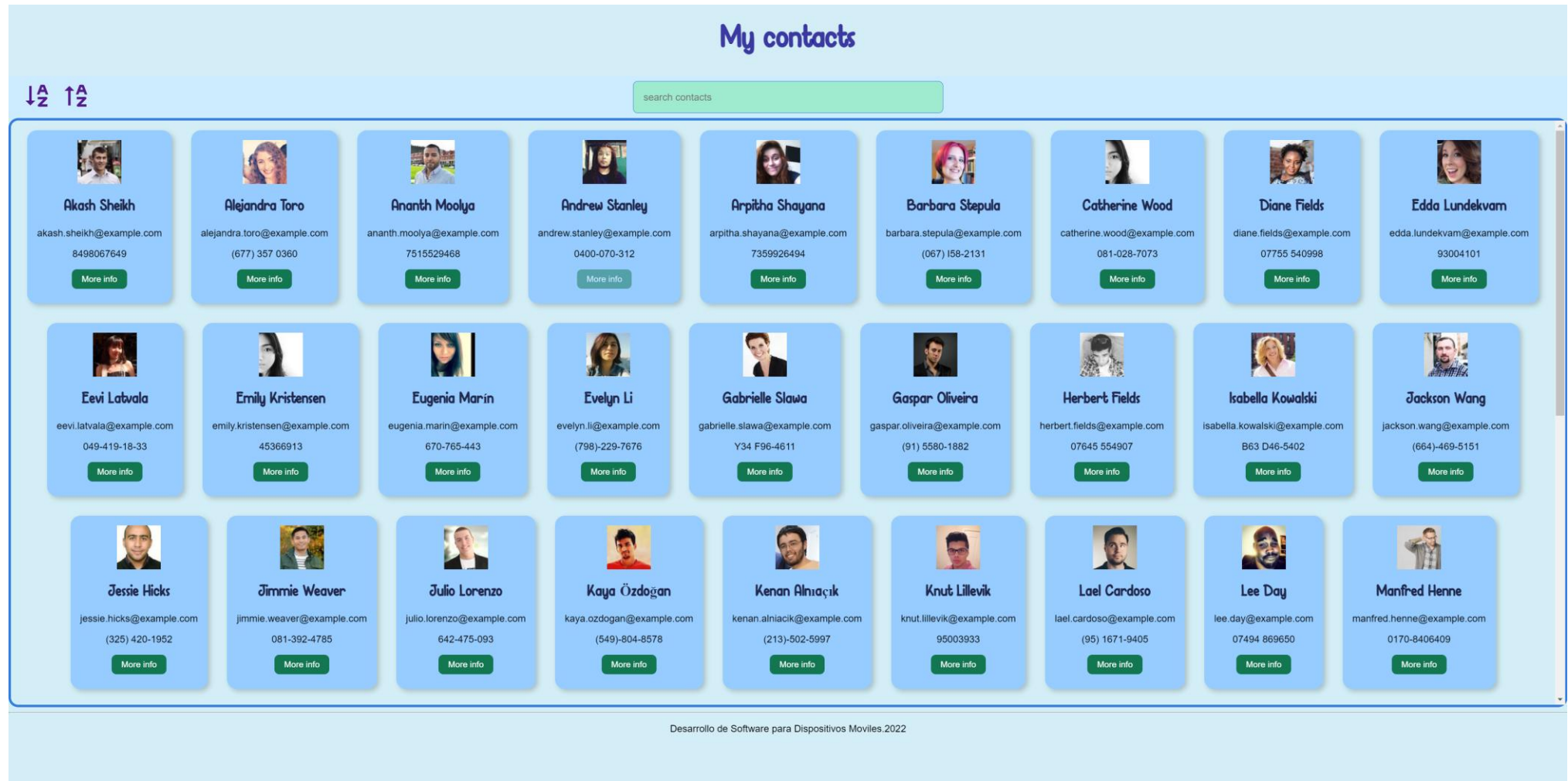
- React se basa en el concepto de **separation of concerns** (SoC)



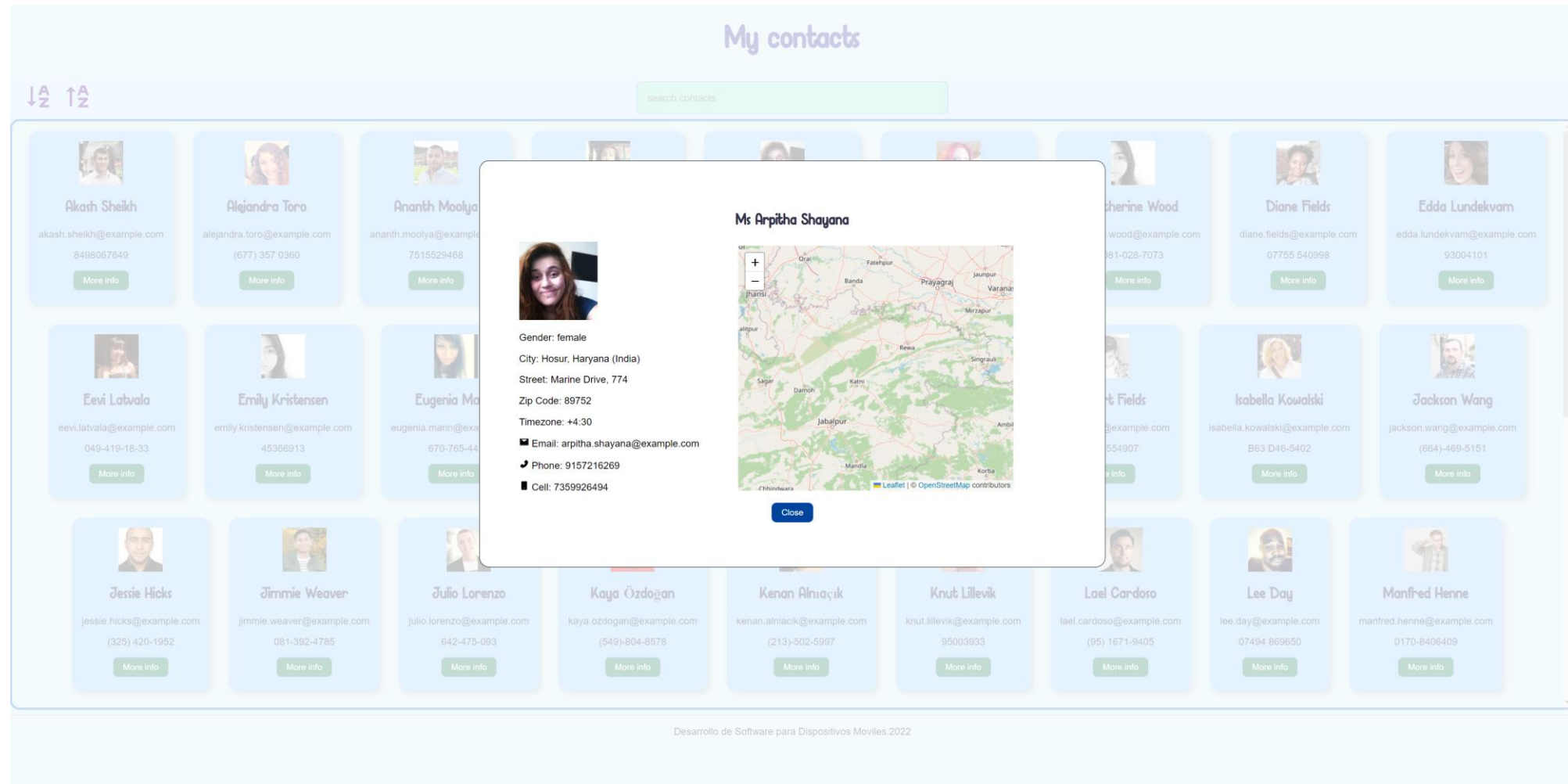
Ecosistema de React



Aplicación que vamos a construir



Aplicación que vamos a construir (II)



REACT

Prerrequisitos de trabajo

Node

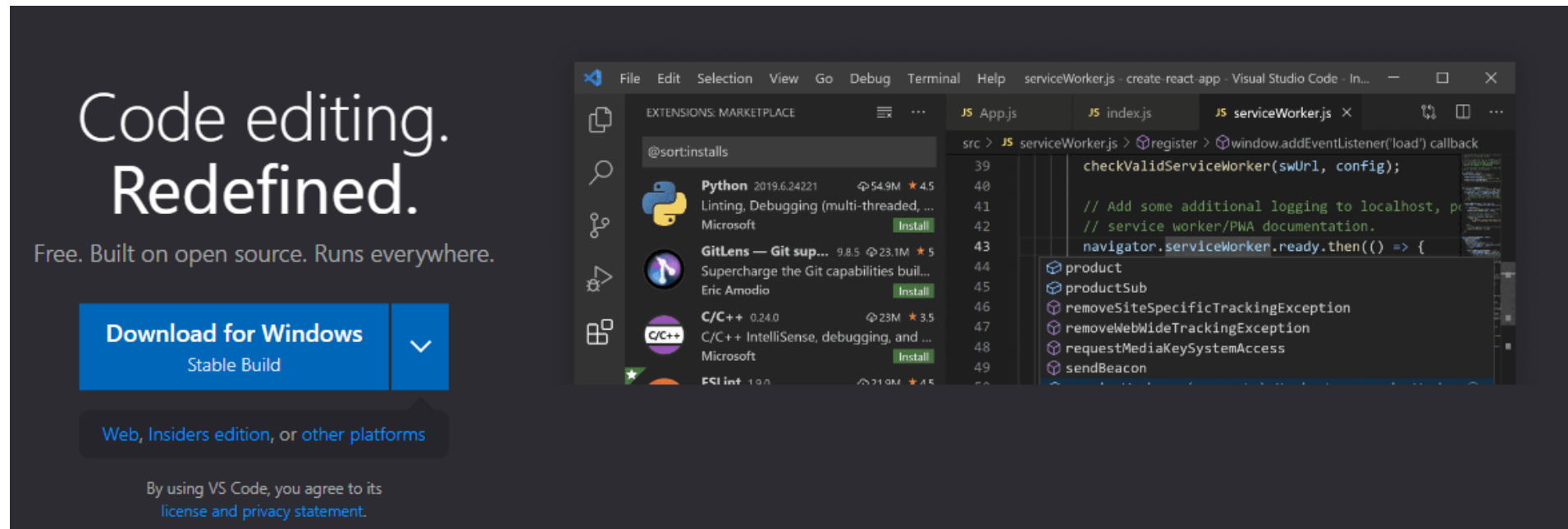
- Entorno de ejecución de JavaScript
 - También instala el gestor de paquetes **npm**
- <https://nodejs.org/en/>
- Mi versión: **24.12** (febrero 2026)

```
node --version
```

```
1  module.exports = function(data){
2    if (!data || data.length < 1) return {};
3
4    let d = {},
5        keys = Object.keys(data);
6
7    for (let i = 0; i < keys.length; i++) {
8      let key = keys[i],
9          value = data[key],
10         current = d,
11         keyParts = key.split('.');
12
13       for (let index = 0; index < keyParts.length; index++){
14         let k = keyParts[index];
15         if (index >= keyParts.length - 1){
16           current[k] = value;
17         } else {
18           if (!current[k]) current[k] = !isNaN(keyParts[index + 1]) ? [] : {};
19           current = current[k];
20         }
21       }
22     }
23   }
```

Visual Studio Code

- Entorno de desarrollo
- <https://code.visualstudio.com/>
- Mi versión: **1.114** (abril 2026)



Git y GitHub

- Sistema de gestión de versiones distribuido
- <https://git-scm.com/>
- Mi versión: **2.52** (febrero 2026)

```
git --version
```

```
git update-git-for-windows
```



Git is a **free and open source** distributed version control system designed to handle everything from small to very large projects with speed and efficiency.

Git is **easy to learn** and has a **tiny footprint with lightning fast performance**. It outclasses SCM tools like Subversion, CVS, Perforce, and ClearCase with features like **cheap local branching**, convenient **staging areas**, and **multiple workflows**.



REACT

Hola Mundo

Creación de un proyecto

- Crear un proyecto en la carpeta correspondiente

```
npx create-react-app react-hello-world  
npm create vite@latest  
cd react-hello-world
```

- Abrir la carpeta con Visual Studio Code

```
code .
```

- Ejecutar el proyecto en el servidor de desarrollo

```
npm start  
npm install  
npm run dev
```



Vite + React

count is 3

Edit `src/App.jsx` and save to test HMR

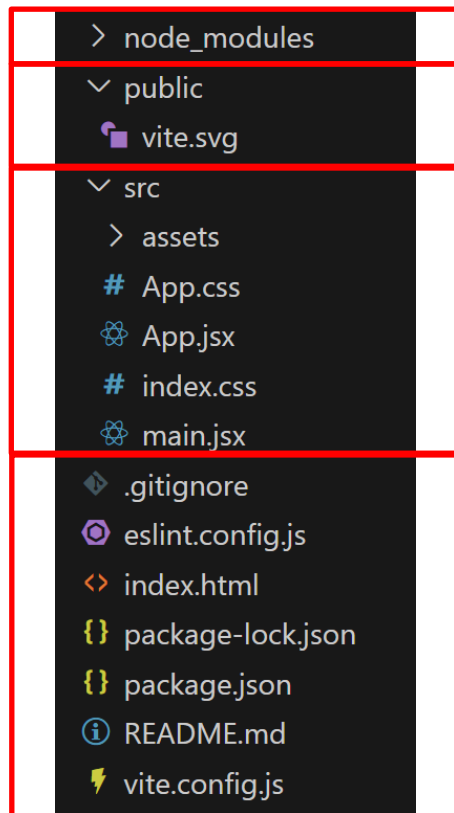
Click on the Vite and React logos to learn more

<http://localhost:5173/>



Estructura del proyecto

➤ Los archivos se dividen en 4 bloques



Guarda todos los paquetes necesarios para el desarrollo

} Aplicación

Punto de entrada “dinámico” de la aplicación

Archivos ignorados por GIT

Punto de entrada de la aplicación

} Dependencias y configuraciones básicas

¡Hola Mundo!

- Archivo **src/App.jsx**
- Es la función principal de la aplicación

```
import logo from './assets/react.svg';
import './App.css';

function App() {
  return (
    <div className="App">
      <header className="App-header">
        <img src={logo} className="App-logo" alt="logo" />
        <p>Edit <code>src/App.js</code> and save to reload.</p>
        <a className="App-link" href="https://reactjs.org">Learn React</a>
      </header>
    </div>
  );
}

export default App;
```



Creación de un componente

- Archivo `src/Message.jsx`
- Pequeño componente que devuelve el texto en una etiqueta **H1**

```
function Message() {  
  return (  
    <h1>Hello World from React</h1>  
  );  
}  
  
export default Message;
```



Importación de un componente desde otro

- Archivo **src/App.jsx**
- Vamos a importar y a utilizar **Message** desde el componente App

```
import Message from './Message';  
  
function App() {  
  return (  
    <Message />  
  );  
}  
  
export default App;
```

src/index.css → vacío



Creación de estilos

➤ Bootstrap para React

```
npm install react-bootstrap  
npm install bootstrap
```

```
"dependencies": {  
  "react-bootstrap": "^2.10.10",  
},
```

➤ Archivo src/Message.jsx

```
import Alert from "react-bootstrap/Alert";  
import './Message.css';  
  
function Message() {  
  return (  
    <div>  
      <h1>Hello World from React</h1>  
      <Alert variant='info'>  
        Hope you enjoy!  
      </Alert>  
    </div>  
  );  
}  
  
export default Message;
```

src/Message.css

```
h1 {  
  background-color: green;  
  color: white;  
}
```

src/main.jsx

```
import 'bootstrap/dist/css/bootstrap.css';
```

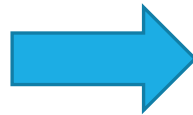
Envío de datos a través de propiedades

- Las **propiedades** o **props** son valores de solo lectura disponibles para una instancia de un componente
- Se pueden establecer mediante **programación** o con la **sintaxis HTML**
- Hay que tener en cuenta que:
 - No se limitan a cadenas o valores primitivos
 - Están disponible dentro del componente
 - Pueden tener tantos elementos como se quiera
- Se utiliza **this.props** para acceder a los diferentes valores

Propiedades de un componente

- En **src/App.jsx**
- Inicializamos el componente **Message** con una propiedad denominada **text**

```
import Message from './Message';  
  
function App() {  
  return (  
    <Message />  
  );  
}  
  
export default App;
```



```
import Message from './Message';  
  
function App() {  
  return (  
    <Message text="Hello people"/>  
  );  
}  
  
export default App;
```



Paso de una propiedad

- En `src/Message.jsx`
- En el componente se pueden acceder a todas las propiedades recibidas

```
import './Message.css';
import Alert from 'react-bootstrap/Alert';

function Message() {
  return (
    <div>
      <h1>Hello World from React</h1>
      <Alert variant='info'>
        Hope you enjoy!
      </Alert>
    </div>
  );
}

export default Message;
```



```
import './Message.css';
import Alert from 'react-bootstrap/Alert';

function Message(props) {
  return (
    <div>
      <h1>{props.text}</h1>
      <Alert variant='info'>
        Hope you enjoy!
      </Alert>
    </div>
  );
}

export default Message;
```



Estado de un componente

- El **estado** son los datos que se pueden cambiar y cuyo historial es mantenido por el componente
- Si un **valor puede cambiar**, es muy posible que **ese valor deba formar parte del estado de la aplicación**
- Valores actualizados con un formulario, datos que vienen de un servidor, tareas que se estaban haciendo en background, etc.
- Se utiliza **useState** para inicializar un estado con su variable asociada y una función para cambiarle el valor

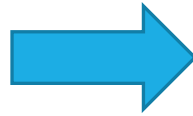
Almacenamiento de un estado

- Archivo **src/App.jsx**
- Un cambio de estado recrea el componente

```
import Message from './Message';

function App() {
  return (
    <Message text="Hello people"/>
  );
}

export default App;
```



```
import {useState} from "react";
import Message from './Message';

function App() {
  const [value, setValue] = useState(1);

  return (
    <div>
      <Message text="Hello people"/>
      <button onClick={() => setValue(value + 1)}>Increment Value {value}</button>
    </div>
  );
}

export default App;
```

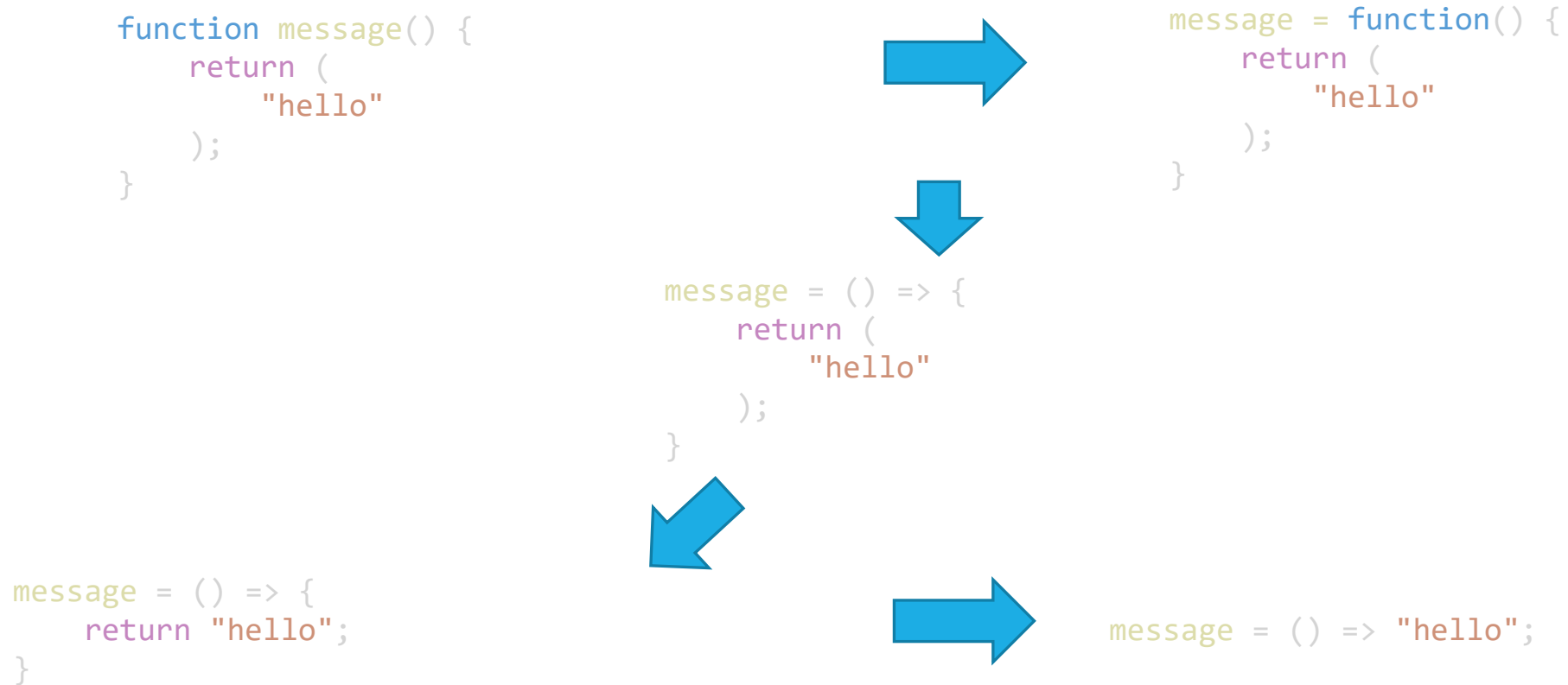


REACT

Conceptos JavaScript que usaremos

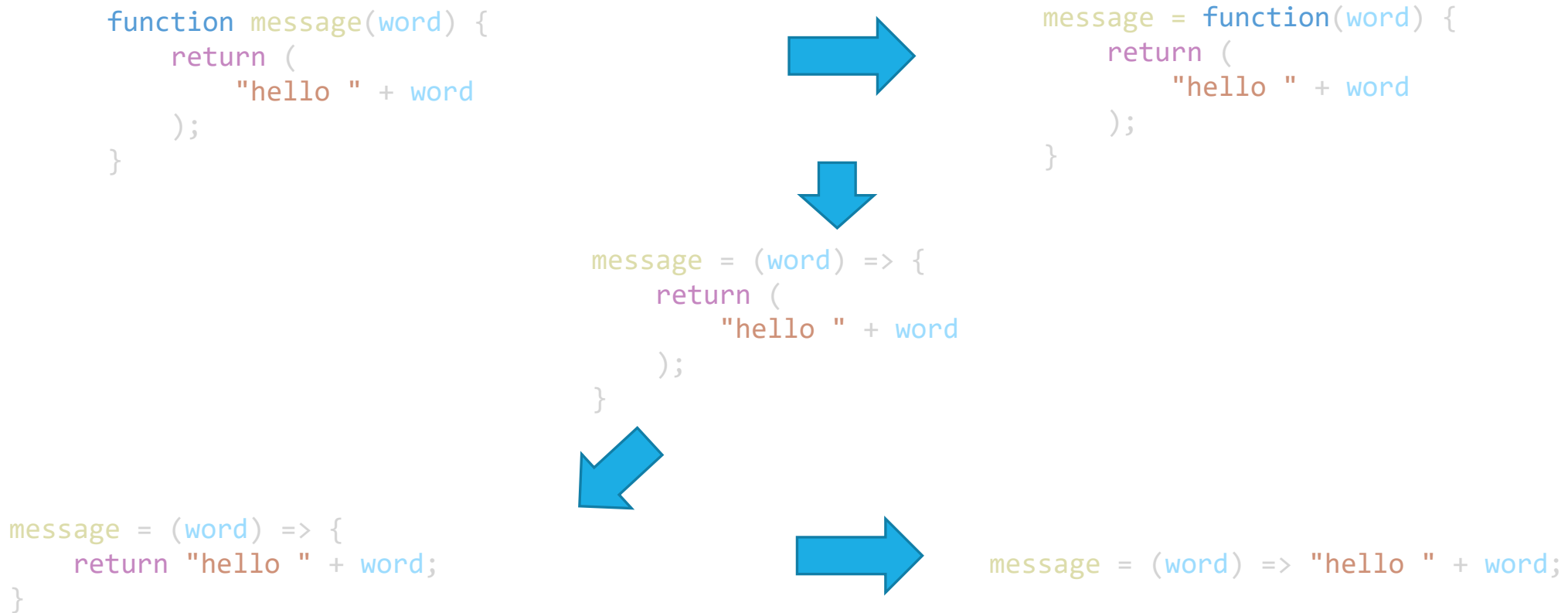
Arrow Functions

- Es un modo de crear funciones en JavaScript escribiendo menos código



Arrow Functions (II)

- Es un modo de crear funciones en JavaScript escribiendo menos código



Destructuring

- Básicamente es una expresión que permite “sacar” valores de propiedades de objetos y asignarles variables independientes

```
const Card = (props) => {  
  return (  
    <div>  
      <p>{props.name}</p>  
      <p>{props.email}</p>  
      <p>{props.phone}</p>  
    </div>  
  );  
}  
  
export default Card;
```



```
const Card = ({name, email, phone}) => {  
  return (  
    <div>  
      <p>{name}</p>  
      <p>{email}</p>  
      <p>{phone}</p>  
    </div>  
  );  
}  
  
export default Card;
```

Maps

- Crea un nuevo array con los elementos resultantes de llamar la función pasada a cada elemento del array original

```
contacts.map((contact, i) => {  
  return <Card  
    key={i}  
    name={contact.name.first}  
    email={contact.email}  
    picture={contact.picture.large}  
  />  
})
```

Filtros

- Crea un “shallow copy” con los elementos resultantes de llamar la función pasada a cada elemento del array original (siempre y cuando la función devuelva **true** para el elemento)

```
contacts.filter(contact => {  
    return (contact.name.first.toLowerCase().includes("peter"))  
});
```

Ordenación

- Ordena los elementos del array y devuelve una referencia al mismo array ordenado

```
contacts.sort((a, b) => {  
    return (a.name.first.localeCompare(b.name.first))  
})
```

Promesas

- Sirven para manejar la potencial respuesta a una operación asíncrona (devuelven promesas que esperan resultados)

```
fetch('https://randomuser.me/api/?results=50')  
  .then(function(response) {  
    return response.json();  
  })  
  .then(function(data) {  
    setContacts(data.results);  
  })  
  .catch(function(error) {  
    console.error('Error: ', error);  
  })
```



```
fetch('https://randomuser.me/api/?results=50')  
  .then((response) => {  
    return response.json();  
  })  
  .then((data) => {  
    setContacts(data.results);  
  })  
  .catch((error) => {  
    console.error('Error: ', error);  
  })
```



```
fetch('https://randomuser.me/api/?results=50')  
  .then(response => response.json())  
  .then(data => setContacts(data.results))  
  .catch(error => console.error('Error: ', error))
```

Named Export VS Default Export

➤ JavaScript proporciona dos formas de exportar elementos de un fichero:

➤ **Named export**

- Se pueden exportar varios elementos con nombre en cada archivo
- Para importarlos en otro archivo se utiliza el mismo nombre
 - Los nombres tienen que indicarse entre llaves {}

```
export function greeting() {  
  console.log('Hello, World!');  
}  
  
// more than one export  
export const bestMovieSeries = 'The Lord of the Rings Trilogy';  
  
// importing in another file  
import { greeting, bestMovieSeries } from './greeting';  
  
greeting(); // Output: 'Hello, World!'  
  
console.log(bestMovieSeries); // Output: 'The Lord of the Rings Trilogy'
```

➤ **Default export**

- Se puede exportar un elemento por defecto en cada archivo
- Para importarlo en otro archivo se puede utilizar cualquier nombre
 - Los nombres no se indican entre llaves {}

```
export default function greeting() {  
  console.log('Hello, World!');  
}  
  
// in another file  
import anyNameWillWork from './greeting';  
  
anyNameWillWork(); // Output: 'Hello, World!'
```

<https://www.bundleapps.io/blog/use-named-exports-over-default-exports-in-javascript>

REACT

Componentes React que usaremos

React Icons

- Componente con una gran cantidad de iconos
- <https://react-icons.github.io/react-icons/>

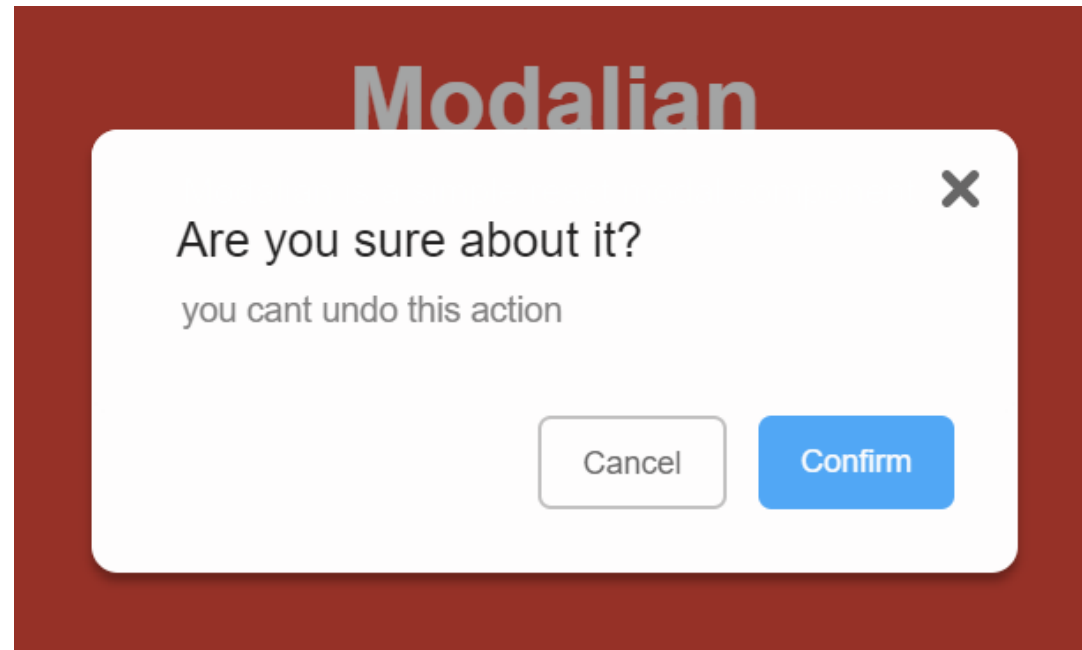
Icon Library	License	Version	Count
Circum Icons	MPL-2.0 license	a2924cb1ee37b9fa39ef023a36f1c884b3492e9b	285
Font Awesome	CC BY 4.0 License	5.15.4	1612
Ionicons 4	MIT	4.6.3	696
Ionicons 5	MIT	5.5.0	1332
Material Design icons	Apache License Version 2.0	4.0.0-12-g63c5cb3060	3650
Typicons	CC BY-SA 3.0	2.1.2	336
Github Octicons icons	MIT	8.5.0	184
Feather	MIT	4.28.0	286
Game Icons	CC BY 3.0	12920d6565588f0512542a3cb0cdfd36a497f910	4040
Weather Icons	SIL OFL 1.1	2.0.12	219
Devicons	MIT	1.8.0	192
Ant Design Icons	MIT	4.2.1	789
Bootstrap Icons	MIT	1.5.0	1846
Remix Icon	Apache License Version 2.0	2.5.0	2271
Flat Color Icons	MIT	1.0.2	329
Grommet-Icons	Apache License Version 2.0	4.6.2	615
Heroicons	MIT	1.0.4	460
Heroicons 2	MIT	2.0.8	530
Simple Icons	CC0 1.0 Universal	5.16.0	2024
Simple Line Icons	MIT	2.5.5	189
IcoMoon Free	CC BY 4.0 License	d006795ede82361e1bac1ee76f215cf1dc51e4ca	491
BoxIcons	CC BY 4.0 License	2.0.9	757
css.gg	MIT	2.0.0	704
VS Code Icons	CC BY 4.0	0.0.23	383
Tabler Icons	MIT	1.68.0	1978
Themify Icons	MIT	v0.1.2	352
Radix Icons	MIT	@radix-ui/react-icons@1.0.3-30-g237cd76	318

React Error Boundary

- Los **Error Boundaries** permiten capturar errores JavaScript en cualquier componente y gestionarlos
- **React Error Boundary** es un componente que implementa la funcionalidad
- <https://github.com/bvaughn/react-error-boundary>
- Solo hay que “envolver” a los componentes que se quieran controlar
- ...e implementar un componente que gestione el error

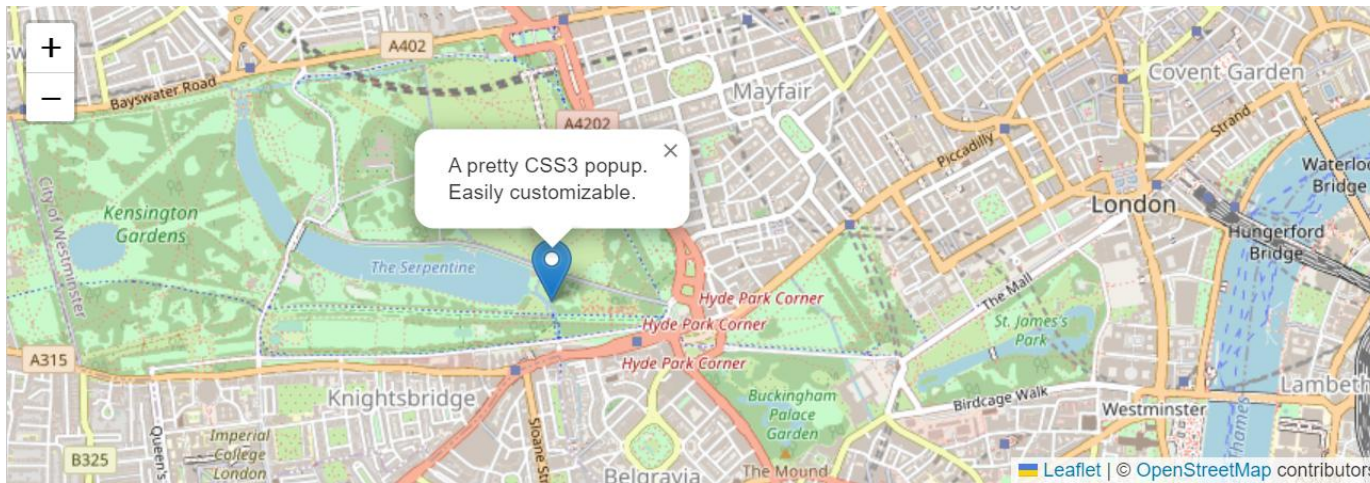
React Modal

- Componente que permite crear ventanas modales fácilmente
- <https://github.com/reactjs/react-modal>



React Leaflet

- Leaflet es una librería open source para trabajar con mapas interactivos
- <https://leafletjs.com>



REACT

Otras
herramientas
que usaremos

Hooks

- Son un añadido a React 16.8
- Son funciones que permiten trabajar con estados y con el ciclo de vida de React sin necesidad de utilizar clases
- Motivación:
 - Hacer el trabajo más fácil para el desarrollador
 - Hacer el trabajo más fácil para la máquina
- Algunos ejemplos:
 - State Hook (useState)
 - Effect Hook (useEffect)



Effect Hook

- Tiene la misma funcionalidad que métodos como **componentDidMount()** o **componentDidUpdate()**
- Permite controlar aspectos externos a la función (recibimos datos, cambiamos el DOM manualmente, recibimos eventos externos, etc.)
- Por defecto se ejecuta cada vez que el componente invoca a **render()**

```
useEffect(() => {  
  fetch('https://randomuser.me/api/?results=50')  
    .then(response => response.json())  
    .then(data => setContacts(data.results));  
})
```



```
useEffect(() => {  
  fetch('https://randomuser.me/api/?results=50')  
    .then(response => response.json())  
    .then(data => setContacts(data.results));  
}, [])
```


Versiones en package.json

➤ ¿Cómo se incrementan las versiones de los paquetes?

Situación del código	Fase	Regla	Ejemplo
Lanzamiento	<i>Nuevo producto</i>	Comenzar con 1.0.0	1.0.0
Solución de bug con compatibilidad hacia atrás	<i>Lanzamiento de parche</i>	Incrementar el tercer dígito	1.0.1
Nueva característica con compatibilidad hacia atrás	<i>Lanzamiento menor</i>	Incrementar el dígito del medio y reiniciar el tercer dígito	1.1.0
Cambios sin compatibilidad hacia atrás	<i>Lanzamiento mayor</i>	Incrementar el primer dígito y reiniciar los dos últimos dígitos	2.0.0

```
"dependencies": {  
  "@capacitor/android": "^4.6.0",  
  "@capacitor/core": "^4.6.0",  
  "@capacitor/ios": "^4.6.0",  
  "@testing-library/jest-dom": "^5.16.5",  
  "@testing-library/react": "^13.4.0",  
  "@testing-library/user-event": "^13.5.0",  
  "react": "^18.2.0",  
  "react-dom": "^18.2.0",  
  "react-scripts": "5.0.1",  
  "tachyons": "^4.12.0",  
  "web-vitals": "^2.1.4"  
},
```

Versiones en package.json (II)

➤ ¿Cómo se especifican posibles versiones de los paquetes?

Valor de ejemplo	Significado
~versión	Versión aproximadamente equivalente. Solo acepta cambios en el parche
^versión	Versión compatible. Acepta cambios menores y en el parche
versión	Versión exacta
>versión	Debe ser una versión mayor
>=versión	Debe ser una versión mayor o igual
<versión	Debe ser una versión menor
<=versión	Debe ser una versión menor o igual
1.2.x	1.2.0, 1.2.1, etc.
1.x.x	1.1.5, 1.9.99, etc.
*	Cualquier versión
latest	La última versión disponible

```
"dependencies": {  
  "@capacitor/android": "^4.6.0",  
  "@capacitor/core": "^4.6.0",  
  "@capacitor/ios": "^4.6.0",  
  "@testing-library/jest-dom": "^5.16.5",  
  "@testing-library/react": "^13.4.0",  
  "@testing-library/user-event": "^13.5.0",  
  "react": "^18.2.0",  
  "react-dom": "^18.2.0",  
  "react-scripts": "5.0.1",  
  "tachyons": "^4.12.0",  
  "web-vitals": "^2.1.4"  
},
```

¿npm install?

Tachyons

- Es básicamente una librería CSS “responsable”
- <https://tachyons.io/>
- Se basa en el principio de “clases que solo hacen una cosa”
- Ejemplo:

- **bw0**: borde con tamaño 0
- **br2**: borde con radio 2
- **bg-dwyl-teal**: color de fondo “dwyl teal”
- **pa2**: padding con tamaño 2
- **white**: color de texto blanco
- **fw1**: ancho de la fuente 1
- **tc**: texto centrado
- **ttu**: transformación del texto a mayúsculas
- **tracked**: con espaciado entre letras

```
<button class="bw0 br2 bg-dwyl-teal pa2 white fw1 tc ttu tracked">do what you love</button>
```



DO WHAT YOU LOVE

Un taquión (del griego ταχυόνιον takhyónion de ταχύς takhýs 'rápido, veloz') es toda aquella partícula hipotética capaz de moverse a velocidades superlumínicas. A los taquiones se les atribuyen propiedades inexplicables a día de hoy

<https://es.wikipedia.org/wiki/Taqui%C3%B3n>

Cufonfonts

- Tipos de letras gratuitos para descargar y usar en la aplicación
- <https://www.cufonfonts.com/>
- Tienen diferentes formatos: TTF, OTF, WOFF, etc.

Boudin Thin

Added by Paolo Arfelli (12 Styles)

Boudin Thin Regular AaBbCcDdEeFfGgHhIiJj

Font-Face Web fonts & TTF-OTF

[Download](#)

[+ Add to List](#)

Delaunay Thin

Added by Paolo Arfelli (12 Styles)

Delaunay Thin Regular Aa

Font-Face Web fonts & TTF-OTF

[Download](#)

[+ Add to List](#)

Depero Book

Added by Paolo Arfelli (2 Styles)

Depero Book Regular AaBbCcDdEe

Font-Face Web fonts & TTF-OTF

[Download](#)

[+ Add to List](#)

William Jameson Font

Added by Rinaldo Hasibuan (1 Style)

William Jameson Free Regular Aa

Font-Face Web fonts & TTF-OTF

[Download](#)

[+ Add to List](#)

A place in the Sun

Added by Paolo Arfelli (1 Style)

A PLACE IN THE SUN REG

Font-Face Web fonts & TTF-OTF

[Download](#)

[+ Add to List](#)

GitHub Pages

- Permite **desplegar** la aplicación directamente del repositorio GitHub
- <https://pages.github.com/>
- Limitaciones:
 - No sirve para sitios comerciales
 - 1 GB de espacio en disco
 - 100 GB de ancho de banda por mes
 - 10 builds por hora

